Property valuation and ranking: training feature comparison

Ptixiaki project vs nineframes-eiv-worker archive | price trees, verified sales, and priority ranking

The database tables are similar, but the training matrices are not. The biggest differences are the sale-price target, sparse verified-sale rows, raw-vs-engineered features, and the archive's separate listing-exit model.

What each system trains

	Your project	Archive
Price model	XGBoost Phase 1 and Phase 2 per property segment and sell/rent. Direct EUR/m2 target from listing asking price.	XGBoost Phase 1 and Phase 2 per segment. log1p(EUR/m2) target; expm1 at inference. Scraped listing targets are premium-adjusted toward market value.
Sale rows	Listing rows only in the training query. The current gov_verified_sales table is not unioned into training.	UNION ALL adds eligible gov_verified_sales to sell segments except residence buildings. verified_sale_price is placed in the asking_price/target slot.
Rank model	No separately trained rank tree. Elo combines global residual z, local residual z, and geo-premium z.	Separate XGBoost AFT model estimates listing exit within 30/60/90 days. Final priority: 70% value percentile + 30% exit-speed percentile.

What the shared schema does and does not imply

Both read properties, base_attributes, location and type-specific detail tables. In the archive, a valuations INNER JOIN is an eligibility gate for all price segments, including residences; for sell listings it also requires v.asking_price > 1000. The actual selected target is p.asking_price, not a valuations column. Your residence query has no valuations join; your land/commercial queries do. Matching table names therefore do not mean matching training rows.

The archive also calls get_market_premium_for_point for listings. It divides each scraped asking price by 1 + premium/100 before forming the sale-price target; verified sale prices stay unchanged. IsVerifiedSale and AreaPremiumPercent are dropped from the price-tree input matrix. The adjustment still changes the target and the rows' residuals.

Verified-sale limitation

The archive's government-sale branch supplies area, location, subtype, zone value, sometimes construction year or city-plan status, and sale price. It explicitly sets many building, amenity, view and facade fields to NULL. These rows enter the same sale tree with no extra price-model sample weight. XGBoost can route missing values, but the model may learn the source-specific missingness pattern rather than a comparable property signal. The code does not establish how large the resulting bias is; that needs a held-out sale evaluation.

Sources: project QueryBuilder.go, ConfigureTree.go, Tree.py, TypeConverter.go, schema_dump.sql; archive rules.go, configs.go, training/train.py, training/train_priority.py, priority.go.

Residence properties: every Phase 2 price-tree feature

Sell and rent have the same named input set. The project has a frozen 30-feature contract; the archive's raw SQL path yields 24 columns after configured drops plus GeoPremium.

Group	Exact feature names	Interpretation
Same name in both (13)	SubType, AdminName, SquareMeters, HeatFrom, EnergyPerformance, AC, Parking, StorageRoom, AluminumFrames, Bedrooms, Pool, Garden, GeoPremium	Name overlap does not ensure equal category mappings or missing-value treatment.
Same name, different form (1)	FloorLevel	Project: parsed numeric lowest floor. Archive training: raw text-array from SQL, then generic numeric coercion; inference: parsed numeric floor. This is a train/inference mismatch.
Project only (16)	Age, RenovAge, NoView, ViewForest, ViewMountain, ViewSea, ViewUnlimited, NoFacade, FacadeCorner, FacadeFrontage, FacadeInterior, FacadeThrough, HeatType, WC, Elevator, FloorCount	Engineered age/renovation, one-hot view/facade, combined bathroom count, elevator and number of floors.
Archive only (10)	ItemType, AdCreationDate, EntryCreatedAt, FacadeType, ConstructionYear, RenovationYear, Heat, ViewType, Bathrooms, Wc	Raw item type, dates and attributes. ItemType/Heat and array fields are not configured categoricals, so generic numeric coercion can turn them into missing values.

The important train/inference contract problem

Archive training runs SQL directly in Python and builds its manifest from the raw query columns. At prediction time, Go constructs an engineered ResidentialFeatures struct. BuildFeatureArray supplies NaN for a manifest name absent from that struct. Thus names such as ConstructionYear, RenovationYear, FacadeType, ViewType, Bathrooms, Wc, ItemType, AdCreationDate and EntryCreatedAt are not reproduced at inference. Conversely, the engineered Age, RenovAge, view/facade indicators and WC never reach this archive price tree. This is an inference from the exact code paths; no live archive model artifact was available to quantify its effect.

The project trains from the same engineered CSV field order that its Go encoder produces. Its residence Phase 2 source and saved sell/rent manifests both specify the same 30 names. The archive's unmatched names are more concerning than the similar database schema suggests.

Source locations: project ml_shared/explain_artifacts.py (feature contract), Worker/types/TypeConverter.go, Worker/Models/TreeCode/Tree.py; archive internal/ranking/rules.go, training/train.py, encoder.go, features.go.

Other segments and the separate priority tree

Price-tree input differences below list every name present on only one side, after configured drops. Names shared by both are omitted for space.

Segment / counts	Only your project	Only archive
Residence building 16 vs 15	Age, RenovAge, NoFacade, FacadeCorner, FacadeFrontage, FacadeInterior, FacadeThrough, HeatType, Elevator	ItemType, SubType, AdCreationDate, EntryCreatedAt, FacadeType, ConstructionYear, RenovationYear, Heat
Landplot (buildable) 24 vs 14	CoverageFactor, NoView, ViewForest, ViewMountain, ViewSea, ViewUnlimited, NoFacade, FacadeCorner, FacadeFrontage, FacadeInterior, FacadeThrough, AngleAmphitheatrical, AngleFlat, AngleSloped, AngleSouth, AngleSouthwest, AngleUnknown	ItemType, SubType, AdCreationDate, EntryCreatedAt, ViewType, FacadeType, CoverageRatio
Agricultural land 22 vs 12	NoView, ViewForest, ViewMountain, ViewSea, ViewUnlimited, NoFacade, FacadeCorner, FacadeFrontage, FacadeInterior, FacadeThrough, AngleAmphitheatrical, AngleFlat, AngleSloped, AngleSouth, AngleSouthwest, AngleUnknown	ItemType, SubType, AdCreationDate, EntryCreatedAt, ViewType, FacadeType
Commercial 30 vs 24	Age, RenovAge, Distance, IsRenovated, AgeSq, InteractionAgeDistance, LogInteraction, HeatType, WC, NoFacade, FacadeCorner, FacadeFrontage, FacadeInterior, FacadeThrough, FloorCount	ItemType, AdCreationDate, EntryCreatedAt, FacadeType, ConstructionYear, RenovationYear, Heat, Bathrooms, Wc

Shared-name encoding differences also matter: the archive trains on raw ViewType/FacadeType arrays and raw FloorLevel arrays where present; your project uses numeric indicators/floor summaries. The archive's raw CoverageRatio is not the project's encoded CoverageFactor. Missing numerics are generally NaN in the archive; your current encoder often uses 0 or -1 sentinels. Dates use a fixed 2026 reference in your encoder but the current year in archive Go code; the archive Python price trainer does not build those age fields at all.

The archive's priority/exit tree: its complete production feature builder

portfolio_age_days, log1p_portfolio_age_days, asking_price_per_sqm, predicted_price_per_sqm, valuation_confidence, square_meters, optional source_publication_age_days, and one-hot item_type=..., transaction=..., subtype=... keys. The last three expand to category-specific columns learned from training snapshots. Its outcome is an interval-censored listing exit; it is not a verified sale label. Training uses lifecycle snapshots, with total weight one per lifecycle. Your Elo has no equivalent trained exit features or tree.

Consequences for numbers

Feature overlap is real, but these differences can move estimated EUR/m2 and ranking positions materially. The archive's 70/30 priority score is also a different quantity from your Elo. Code inspection alone cannot say how large the gap will be on your data; paired predictions on the same held-out properties are needed.

Sources: project ConfigureTree.go, ModeledTypes.go, TypeConverter.go, Tree.py; archive configs.go, rules.go, training/train.py, features.go, encoder.go, priority.go, training/train_priority.py.